

Utilización avanzada de clases II. Polimorfismo

Esteban Álvarez

Polimorfismo

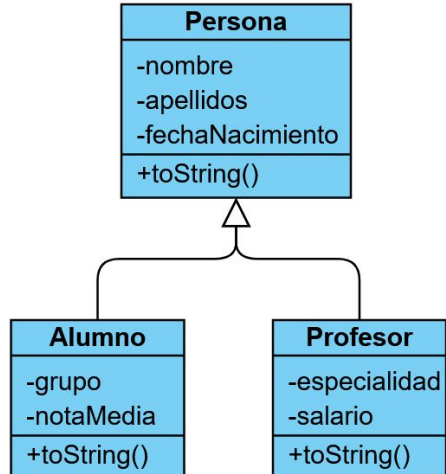
- Cuando vimos listas, conjuntos, hashMaps y arrays comprobamos que sólo se pueden crear de **un tipo de datos**: String o enteros o float...
- Problema: ¿Y si queremos crear una lista de alumnos y profesores?
 - En principio no es posible
 - Tendríamos que crear dos listas: una de alumnos y otra de profesores

```
Alumno[] alumnos=new Alumno[5];
```

```
Profesor[] profesores=new Profesor[5];
```

Polimorfismo

- Hay una **solución**:
 - como ambos Alumno y Profesor son personas
 - se puede crear una lista de personas que incluya alumnos y profesores



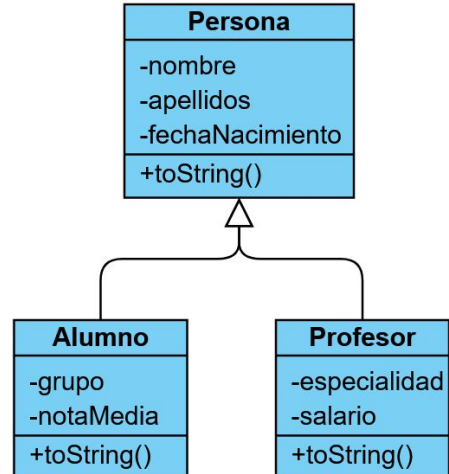
```
Persona[] listaPersonas= {new Alumno("Alumno","Apealum",
                                new LocalDate(2020, 10, 30),
                                "1DAM",9.7),
                            new Profesor("Esterban","Alvarez",
                                new LocalDate(1900, 1, 1),
                                1000,"IFC")};
```

Polimorfismo

- Pero cuidado porque el alumno y el profesor anterior es como si se hubiesen declarado de la siguiente manera:

```
Persona alumno=new Alumno();  
Persona profesor=new Profesor();
```

- Son **objetos de tipo Persona** pero que se crean como alumno y profesor respectivamente
- Lo que implica que los alumnos y profesores que pertenezcan a la lista de Personas **solo pueden usar** los métodos heredados de la superclase y los sobrescritos, nunca los suyos propios

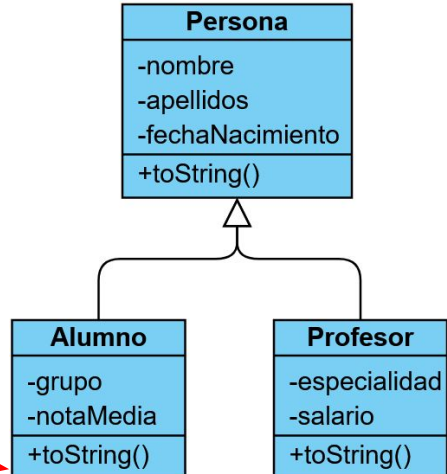


Polimorfismo

- Ejemplo de funcionamiento: mostrar datos con toString():

```
System.out.println(listaPersonas[0].toString());
```

- El primer elemento de la lista es un alumno
- Al llamar al método toString sobre un alumno se muestran sus datos completos al estar **sobrescrito**:
 - Muestra grupo y nota media que son atributos propios de un alumno
 - Y también nombre, apellidos y fecha de nacimiento que son los heredados de persona



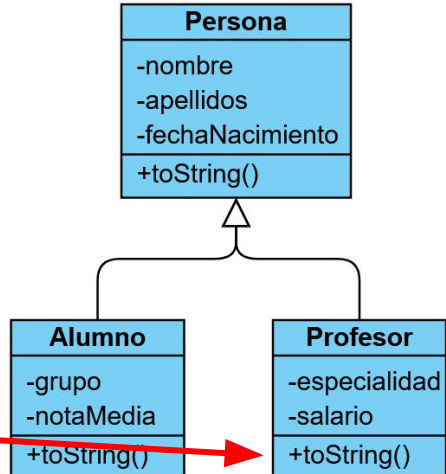
```
Alumno [grupo=1DAM, notaMedia=9.7, toString()=Persona [nombre=Alumno, apellidos=Apealum,
```

Polimorfismo

- Ejemplo de funcionamiento: mostrar datos con toString():

```
System.out.println(listaPersonas[1].toString());
```

- El segundo elemento de la lista es un profesor
- Al llamar al método toString sobre un profesor se muestran sus datos completos al estar **sobrescrito**:
 - Muestra especialidad y salario que son atributos propios de un profesor
 - Y también nombre, apellidos y fecha de nacimiento que son los heredados de persona



```
Profesor [salario=1000.0, especialidad=IFC, toString()=Persona [nombre=Esteban, apellidos=Alvarez
```

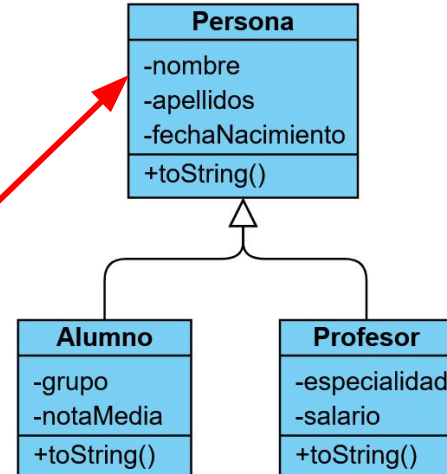
Polimorfismo

- Ejemplo de funcionamiento: uso de métodos

```
System.out.println(listaPersonas[1].getNombre());
```

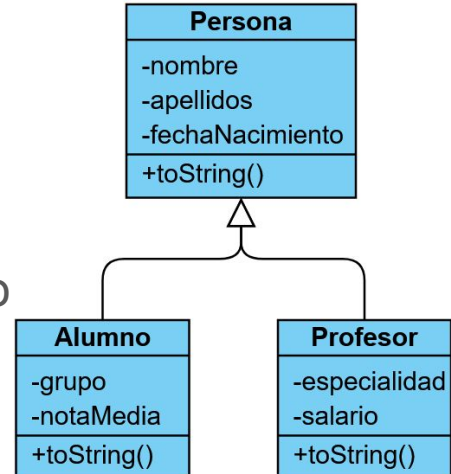
- Al llamar a getNombre de un profesor, se está usando un método heredado de Persona
- Se puede usar porque es un **método heredado**

Esteban

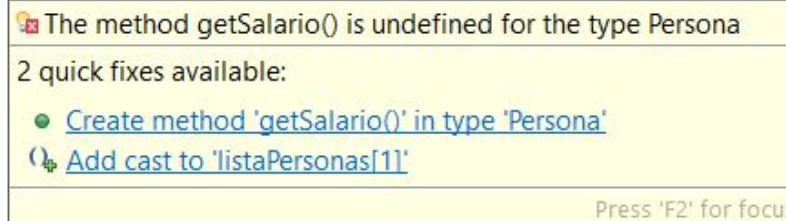


Polimorfismo

- Pero sin embargo, no es posible utilizar el método `getSalario` la clase `Profesor`:



```
System.out.println(listaPersonas[1].getSalario());
```

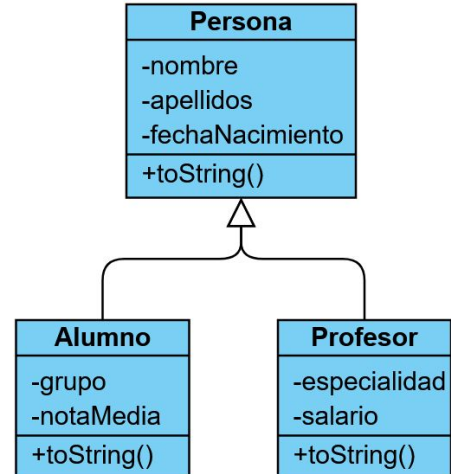


- No se puede usar porque estamos trabajando con un **array de Personas** y no existe el método `getSalario` en esa clase

Polimorfismo

- Se podría añadir un cast a la línea anterior
- Pero esta solución es inviable dentro de un for que recorra el array o la lista

```
System.out.println(((Profesor) listaPersonas[1]).getSalario());
```



- Veremos la solución correcta más adelante con los métodos abstractos

Polimorfismo. Conclusiones

- El polimorfismo **permite** crear colecciones de diferentes tipos de datos
- Un objeto se puede **comportar de diferentes formas** (polimórfico). En el ejemplo anterior:
 - definimos un array de Personas
 - en él podemos añadir alumnos o profesores indistintamente
 - el comportamiento de los métodos que usemos será diferente dependiendo de si se trata de un alumno o un profesor
- El **polimorfismo es posible** gracias a la herencia y a los métodos sobrescritos
- Los **métodos y clases abstractas** también son uno de los pilares del polimorfismo

Polimorfismo

EJERCICIO

- Crea un programa que implemente un ArrayList y un menú de opciones que permita al usuario:
 1. añadir un alumno a la lista. Deberá pedir los datos necesarios al usuario
 2. añadir un profesor a la lista. Deberá pedir los datos necesarios al usuario
 3. mostrar los datos de todos los componentes de la lista
 4. mostrar los datos de un elemento de la lista. Pedir la posición de la lista a mostrar

Polimorfismo

EJERCICIO

[Solución](#)

Métodos abstractos

- Un método abstracto es un método de una superclase cuya implementación no se define, es decir, **no tiene código**
- Solo se declara **su cabecera**
- Las **subclases deben implementar** el código de un método abstracto
- Cada subclase implementa el método **adaptándolo** a sus características

Métodos abstractos

Ejemplo:

- La **clase Animal** puede tener un método emitirSonido() pero el concepto de emitir sonido en un animal es tan **amplio** que no tiene sentido implementarlo por lo que se declara como abstracto (y sin código)
- La **subclase Perro** implementa el método anterior para mostrar por pantalla GUAU, GUAU!
- La **subclase Gato** implementa el mismo método y mostrará MIAAAAAAU!

Métodos abstractos

- La sintaxis es simple: añadir la palabra reservada **abstract** después del modificador de acceso public

```
public abstract void nombreDelMetodo();
```

- La sentencia finaliza con ;
- Se debe especificar el tipo de datos devuelto y los parámetros que recibe (como cualquier método)
- Pero **no tiene código**

Métodos abstractos

- Que una clase tenga métodos abstractos obliga a que esa **clase sea abstracta**

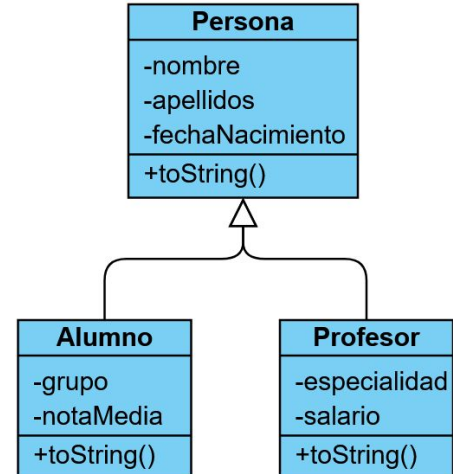
```
public abstract class Persona {
```

- Una clase **puede tener** métodos abstractos y otros que no lo son
- Una **subclase que no implemente** los métodos abstractos que ha heredado, obliga a que lo hagan sus subclases

Métodos abstractos

EJERCICIO

- En el ejercicio de la lista polimórfica vimos que no se podían mostrar los datos del profesor:
`listaPersonas[1].getSalario();`
- Para solucionarlo declara el método abstracto `mostrarDatosPropios()` en la clase `Persona`
- Impleméntalo dentro de las clases `Profesor` y `Alumno` para mostrar sueldo y especialidad para el profesor y grupo y nota media para el alumno
- Muestra los datos propios de cada elemento de la lista



ACLARACIÓN: No es habitual declarar el método `mostrarDatosPropios()` en las subclases para mostrar sus datos. Se hace aquí a efectos didácticos

```
public abstract String mostrarDatosPropios();
```

```
@Override  
public String mostrarDatosPropios() {  
    return "grupo=" + grupo + ", notaMedia=" + notaMedia;  
}
```

```
@Override  
public String mostrarDatosPropios() {  
    return "especialidad=" + especialidad + ", salario=" + salario;  
}
```

Métodos abstractos

EJERCICIO

- Crea el método abstracto mejorar() en la clase Persona
- Que un alumno mejore significa que se aumenta su nota media en un punto (con un máximo de 10 puntos)
- Que un profesor mejore significa que aumenta su sueldo en 100€
- Modifica el ejercicio anterior de la lista polimórfica para mejorar a todos sus componentes, ya sean alumnos o profesores

Métodos abstractos

EJERCICIO

[Solución](#)

Métodos abstractos

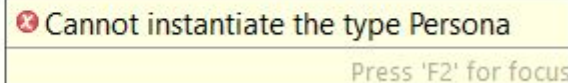
¿CUÁNDO USAR MÉTODOS ABSTRACTOS?

- Cuando una superclase **sabe que va a tener un determinado comportamiento** (Vehículo->Arrancar, Embarcación->Navegar, Figura->CalcularArea) pero ese comportamiento no se puede definir exactamente en esa clase, si no en sus subclases
 - No se arranca igual un coche que una moto que un patinete eléctrico
 - Una lancha de pedales, un velero y un carguero navegan de forma diferente
 - El área de un círculo, un triángulo y un rectángulo se calculan de forma distinta

Clases abstractas

- Que una clase sea abstracta implica que no se pueden crear objetos de ella

```
Persona persona=new Persona();
```



✖ Cannot instantiate the type Persona
Press 'F2' for focus

- Aunque habría la posibilidad de hacerlo siempre y cuando se implementen los métodos abstractos en el momento de la creación del objeto

```
Persona persona=new Persona() {  
  
    @Override  
    public void mejorar() {  
        // TODO Auto-generated method stub  
    }  
};
```

Clases abstractas

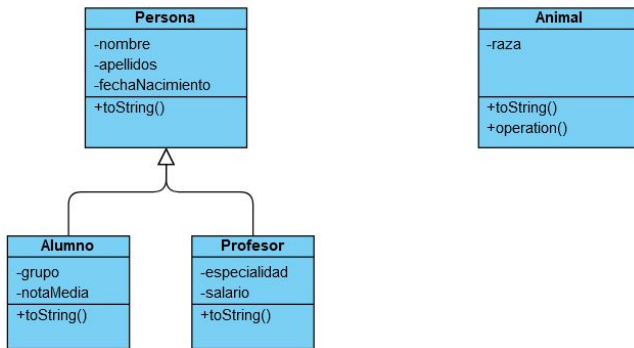
- Ejemplo del click de un botón en Android:

```
boton.setOnClickListener(new View.OnClickListener() {  
    public void onClick(View v) {  
        //onClick es un método abstracto  
        //que es obligatorio implementar  
  
        //Aquí ponemos el código a ejecutar  
        //cuando se haga click al botón  
  
    }  
});
```

Clases abstractas

¿CUÁNDO HACER CLASES ABSTRACTAS?

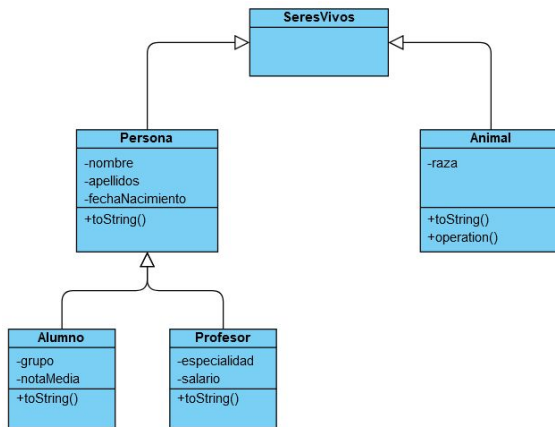
- Ya vimos que al declarar un **método como abstracto** obliga a que la clase sea abstracta
- Pero ¿cuándo **necesitamos** crear una clase abstracta sin más?
- La respuesta es sencilla. Imagínate que ahora tenemos en nuestra aplicación la clase Animal y queremos hacer una lista de profesores, alumnos y animales:



Clases abstractas

¿CUÁNDO HACER CLASES ABSTRACTAS?

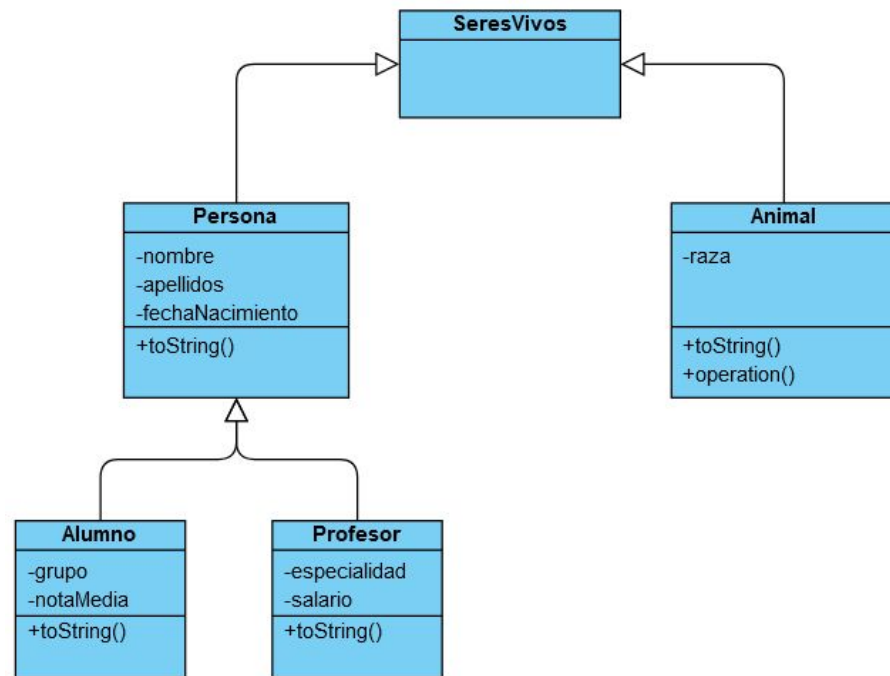
- Tuvimos el mismo problema con profesores y alumnos y la solución fue utilizar la superclase Persona
- Aquí no tenemos una superclase que agrupe personas y animales por lo que **estamos obligados** a crearla, por ejemplo, seres vivos



Clases abstractas

¿CUÁNDO HACER CLASES ABSTRACTAS?

- Puesto que el **único objetivo** de SeresVivos es poder crear una lista polimórfica, no es necesario crear objetos de esa clase y se declara abstracta
- Ahora habría que mirar qué atributos y métodos son comunes a Persona y Animal y por tanto, heredables...



Modificador FINAL. Atributos

- El modificador `final` aplicado a un atributo implica que el valor de ese atributo no se puede modificar, **es constante**
- Las constantes se escriben en **mayúsculas**
- Ejemplo:

```
public abstract class Persona {  
    final int CODIGO=1;  
}
```

- Todas las personas (y sus subclases) tienen el código con valor 1

Modificador FINAL. Métodos

- Un método declarado como final, **no puede ser sobrescrito** (override) por las subclases
- Se puede ver como un método **constante**: se hereda pero no se puede modificar

```
public class Persona {  
    final public String mostrar() {  
        ...  
    }  
}
```

En este ejemplo, las clases hijas de Persona heredan y pueden utilizar el método mostrar() pero no lo pueden sobrescribir (modificar)

Modificador FINAL. Clases

- Un clase final, **no puede tener subclases**

```
public final class Alumno extends Persona {
```

La clase Alumno no puede tener clases derivadas

EJERCICIOS

Hacer las actividades de Polimorfismo del Campus Virtual

FIN!